

L09: Introduction to Language Models

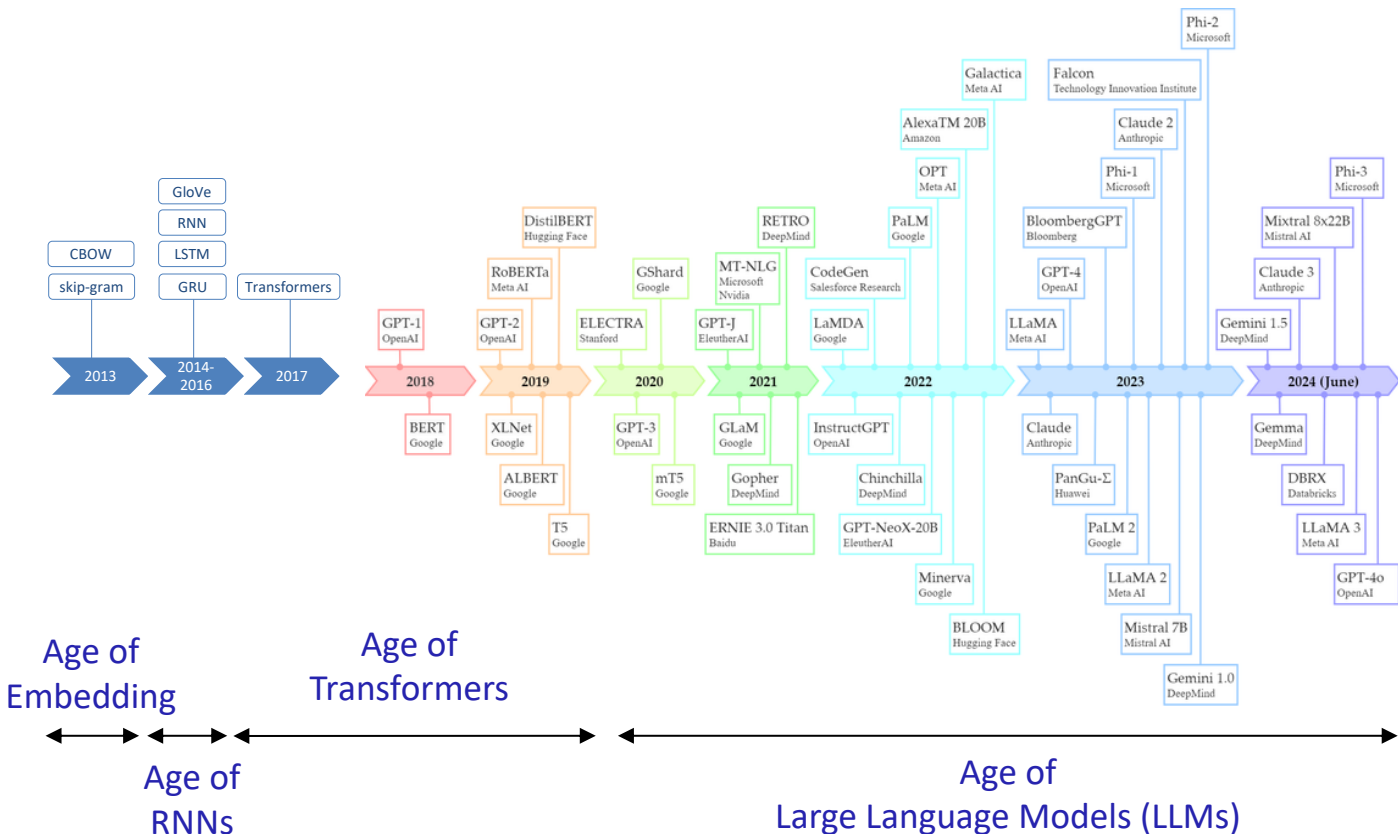
FICT, UTAR

Table of Contents

1. Introduction to Large Language Models
2. What is a Language Model?
3. Pre-training
 - Supervised Fine-Tuning (SFT)
 - Causal Language Modeling (CLM)
 - Evaluation
4. Inference
 - Decoding Strategies
 - Greedy Sampling
 - Top- k and Top- n Sampling
 - Random Sampling with Temperature
 - Repeat Penalty
 - Beam Search

Introduction to Large Language Models (LLMs)

Evolution of Language Models



Adapted from [Hang et al., Future Internet 2024]

Evolution of LLMs

Model	Release Date	Parameters	Number of Layers	Maximum Context Length	Modality	Supported Languages	Training Dataset Size	Other Key Features
GPT-1	June 2018	~117 M	12	512	Text-only	English (primarily)	~5 GB (BookCorpus)	Introduced transformer architecture
GPT-2	Feb 2019	~1.5 B	48	1,024	Text-only	English (primarily)	~40 GB (WebText)	Demonstrated “zero-shot” capabilities
GPT-3	June 2020	~175 B	96	2,048	Text-only	Multilingual (~26+)	~570 GB (Common Crawl, WebText, Books, Wikipedia)	Few-, one-shot learning, widespread adoption
GPT-4	March 2023	Estimated ~1 T+	Estimated 120+	8K–32K	Text-only (GPT-3.5), text+image for GPT-4	Multilingual (expanded)	Undisclosed curated and vision datasets	More robust reasoning, broader generalization
ChatGPT (GPT-3.5 / GPT-4-based)	Nov 2022	~175 B (same as GPT-3.5 / GPT-4)	Same as base model	4K (GPT-3.5), 16K (GPT-4)	Multimodal for GPT-4-based versions	Multilingual (expanded)	GPT-3.5 / GPT-4 datasets with tuning	Fine-tuned for conversational apps
GPT-4 Turbo	Nov 2023	Not specified	—	128K	Multimodal (text + image)	Multilingual (same as GPT-4)	Updated knowledge as of Apr 2023	Much larger context, lower cost, JSON mode
o1	Dec 2024	100B-300B	100+	128K-200K	Multimodal, reasoning (“thinks” before answering)	Multilingual	—	Strong in math, science, programming; more thoughtful responses
o3 / o3-mini / o3-pro	Jan–June 2025	~200B	100+	200K	Multimodal, advanced reasoning with tool use	Multilingual	—	Chain-of-thought style reasoning, very strong benchmarks
GPT-4.5 (“Orion”)	Feb 2025	5-7T (total) ~600B active	Unknown	128K	Multimodal (file/image uploads, canvas)	80+ language	Not specified	Improved hallucination avoidance, creativity, EQ; research preview
GPT-4.1 (incl. mini/nano)	Apr 2025	~1.7-1.8T	~32	128K, 1 million (API)	Multimodal	80+ languages (same as GPT-4.5)	—	Better coding/formatting; lighter versions
GPT-5	Aug 2025	~635B	Unknown	256K	Multimodal (“Thinking” modes, dynamic routing)	Broad language support	—	Unified model replacing earlier ones; quick answer vs deep reasoning modes, personality variants, better alignment and accuracy

Bigger models

Longer context length

Multi-modality support

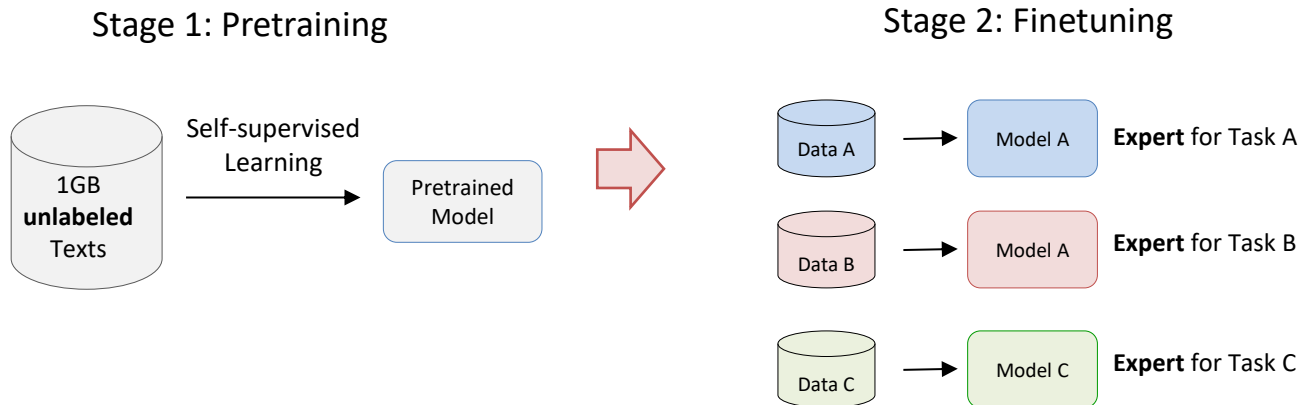
Multi-lingual support

Bigger and more diverse training set

More competent models, development of new skills

GPT1: A Multitask Adaptable Language Model

- GPT1 is not a general language model (generalist), but simply a pretrained model for finetuning **narrow expert**.
- Demonstrates that most NLP task can be generated by generative pre-training **self-supervised learning** followed by **fine-tuning** for downstream tasks.



GPT2: A zero-shot learner

[Radford, A., et al. (2019), [Language Models are Unsupervised Multitask Learners](#)]

- Same structure as but bigger than GPT-1 and trained with more data
- Displays **zero-shot capability** to perform downstream tasks without any finetuning

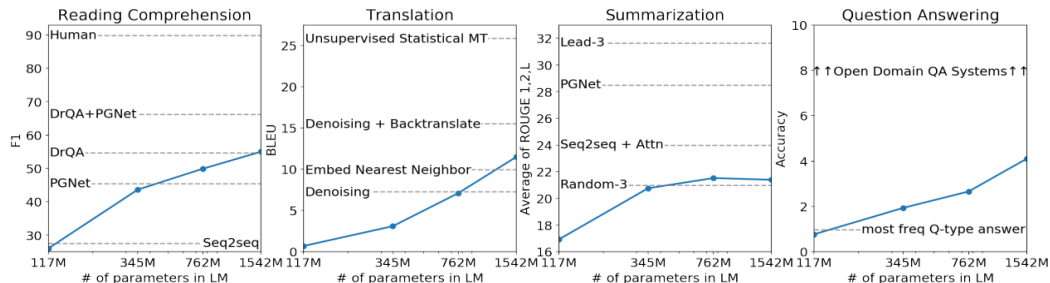


Figure: Zero-shot task performance of GPT-2 as a function of model size on different tasks

- Where does the zero-shot capability comes from?
 - LLMs learn patterns, relationships and statistical representation of input sequence. This **knowledge** is encoded somewhere in its parameters.
 - Does **not** explicitly "**memorize**" facts and knowledge although it may produce verbatim text from training set if the input closely matches training examples, especially highly specific or repetitive ones.

GPT3: Unprecedented Scale (2020)

- Significant increase in model size (**175GB**)
- Trained on Internet-scale dataset (books, wikipedia and web pages)
- Display **in-context learning** abilities to perform a task by observing examples in the **input prompt** without any finetuning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French:  ← task description
2 cheese => .....           ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French:  ← task description
2 sea otter => loutre de mer    ← example
3 cheese => .....             ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French:  ← task description
2 sea otter => loutre de mer    ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => .....             ← prompt
```

Reference: [Language Models are Few-Shot Learners](#) (2020), OpenAI

OpenAI 's Models in Dec 2024 (Proprietary)

Model	Description	Price (Text) (per 1K tokens)	Context	Max Output tokens	Knowledge Cutoff
GPT-4o	Versatile, high-intelligence flagship model. Smarter model but higher price per token	\$0.00250 (input) \$0.00100 (output)	128K	16K	Oct 2023
GPT-4o mini	Fast, affordable small model for focused tasks Not as smart as GPT-4o but Faster model and lower price per token.	\$0.000150 (input) \$0.000600 (output)	128K	16K	Oct 2023
GPT-o1	Trained with reinforcement learning to perform complex reasoning. The model thinks before responding.	\$0.0150 (input) \$0.0600 (output)	200K	100K	Oct 2023
GPT-o1 mini	Smaller but fast and affordable version of GPT-o1	\$0.0030 (input) \$0.0120 (output)	128K	65K	Oct 2023
GPT-4o realtime	Realtime response over WebRTC or a WebSocket interface.	\$0.0050 (input) \$0.0200 (output)	128K	4K	Oct 2023

1000 tokens is about 750 words

Up-to-date as of Dec 2024

Types of Access

	API (Proprietary)	Open-weight	Open-Source
Definition	Access to a model hosted via a service endpoint	Downloadable pre-trained weights available for download	Pre-trained weights and full model (code) available
Usage Freedom	Usually requires a subscription	Restricted by license (non-commercial)	Unrestricted (modify & distribute)
Examples	 ChatGPT, GPT4, GPT4o, o1...  Claude  Gemini	 Meta LLAMA 3  MISTRAL AI_  Qwen  Microsoft Phi-3  Gemma	 EleutherAI  LLM360  starcoder  AI2 OLMo Open Language Model

What is Language Model?

What is a Language Model?

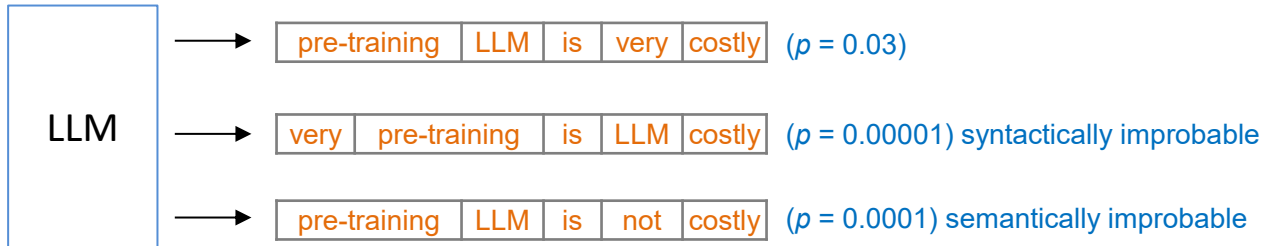
- A language model (LM) computes a **probability distribution** over a sequence of tokens:

$$p(x^{<1>}, \dots, x^{<T>})$$

- LMs are **generative** models - you can sample new sentences from it:

$$x^{<1>}, \dots, x^{<T>} \sim p(x^{<1>}, \dots, x^{<T>})$$

- High-probability sentences follow **common grammatical patterns** and contain **frequently** co-occurring word combinations from the training data.



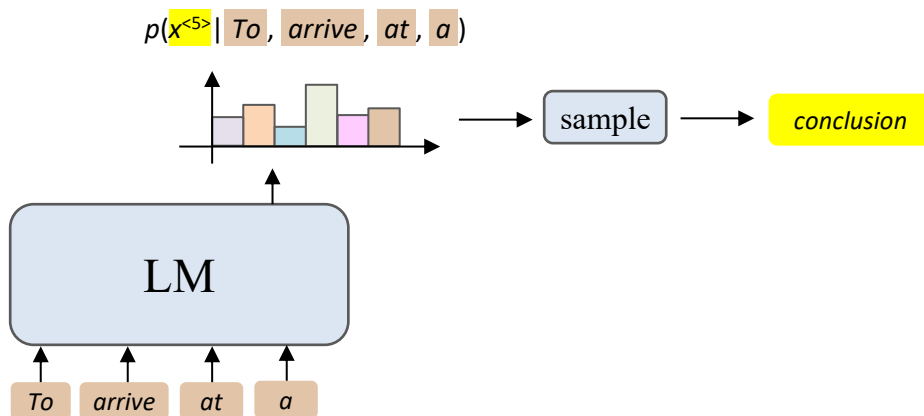
Autoregressive Language Model

- The output of an autoregressive LLM is the **joint distribution** of a token sequence, which factorizes as a product of **conditionals**.

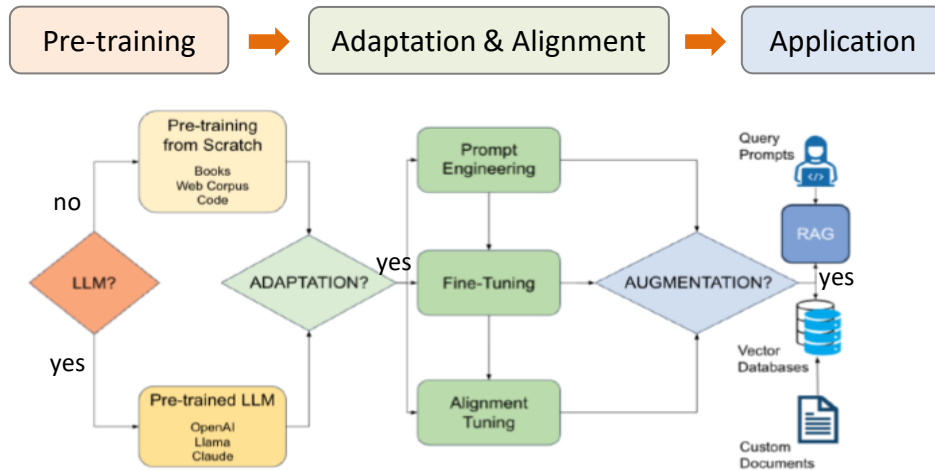
$$p(x^{<1>}, \dots, x^{<T>}) = p(x^{<1>}) p(x^{<2>} | \hat{x}^{<1>}) p(x^{<3>} | x^{<1>}, \hat{x}^{<2>}) \dots$$

- Language model computes the **conditional probability** of the **next** token:

$$p^{<t>} = p(x^{<t>} | \hat{x}^{<1>}, \dots, x^{<t-1>})$$



Construction Pipeline for LLM Systems



- **Pre-training:** pre-training from scratch or choose existing pretrained LLMs
- **Adaptation & Alignment:** adapts the pre-trained LLM to the targeted task
 - **Fine-tuning:** extra training to build a specialist model
 - **Alignment tuning:** align to human values and certain use case, e.g., through RHLF
 - **Prompt engineering:** design prompt template for the targeted application
- **Application:** augments the LLM with external sources by using a Retrieval Augmented Generation (RAG) system to access external knowledge sources.

Pre-training

What matters when pre-training LLMs

- What matters when pre-training LLMs:
 - Architecture
 - Training algorithm & Loss function
 - Compute
 - Data
 - Evaluation
 - Systems
- Current **architecture** (Transformer), **training algorithm** (adam or AdamW) and **loss function** (cross-entropy) are already very effective.
- In practice, focus on training **data**, **evaluation** and **system optimization**.

What pre-trained LLM can do

- Pre-training allows LLM to:
 - Learn **fundamental language skills** (language pattern and grammar)
 - Learn how to **reason**
 - Acquire the **knowledge base** from large-scale datasets
 - Serve as a **foundation model** for downstream tasks (via fine-tuning or prompting)
- Pre-trained model may have the following **limitations**:
 - **Domain knowledge** in certain areas (obscure plant diseases, expert-level medical knowledge, etc.)
 - Specific **skill** (have a proper conversation, follow instructions, etc.)
 - **Ethics** issues (may still generate bias or toxic content, suffer from hallucinations or misinformation, no human values in mind)

Pre-training dataset: Preprocessing

- Data is the key to the success of LLM. Use **internet-scale** data to pre-train the model.
- Data pre-processing:
 - Download the internet.
 - Common crawl 250 billion pages. >1PB (1e6GB)
 - Extract text from HTML.
 - Need to remove boiler plate data and able to handle mathematical expressions
 - Remove harmful content
 - NSFW (Not safe for work), harmful content, PII (Personally Identifiable Information)
 - Remove duplicate url, document or line
 - E.g., headers, footers or menu
 - Filtering
 - Heuristic filtering: remove low quality documents, unreasonably long or short pages
 - Model based filtering: predict if pages is a good reference by Wikipedia
 - Data mix
 - Classify data categories (code/book/entertainment). Reweight the domains (e.g., down-weight entertainment, up-weight code) to get higher downstream performance.

Pre-training dataset: Data source

- **Proprietary** LLMs (GPT-4, Claude, Gemini, etc.) are secretive.
 - Only mentioned "publicly available internet data".
 - Lack specifics about data sources, filtering methods or token counts.
- **Open source** models (LLaMA, DeepSeek, 01.AI, Qwen) are more transparent on the training set and methodology. List of commonly used data:

Web Data

CommonCrawl

Extensive web crawl data, often heavily filtered

C4 (Colossal Clean Crawled Corpus)

Cleaned version of CommonCrawl

OSCAR

Multilingual filtered web data

Academic/Scientific

arXiv

Scientific papers across multiple domains

PubMed

Biomedical literature

Books3

Digital books collection

Size of training set:

- LLaMA2 (2T tokens)
- LLaMA3 (15T tokens)
- GPT-4 (~13T tokens?)

Code

GitHub Public Repositories

Source code across multiple languages

Stack Exchange

Programming Q&A discussions

The Stack

Curated programming discussions and documentation

General Knowledge

Wikipedia

Encyclopedia articles in multiple languages

Project Gutenberg

Public domain books

The Pile

Curated collection of diverse datasets

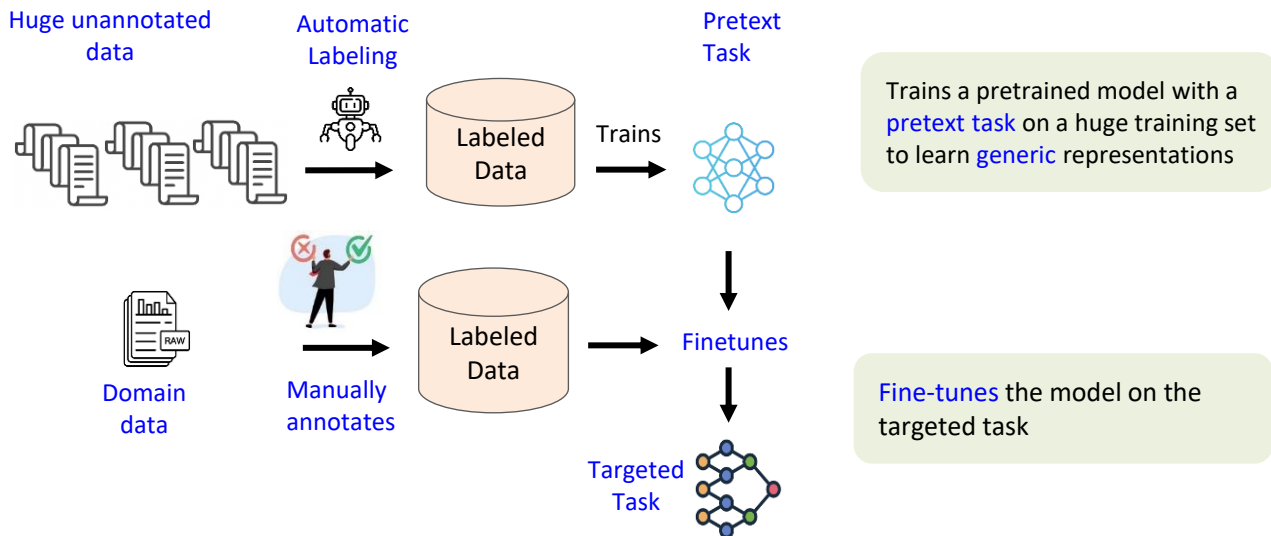
Pre-training: Self-Supervised Learning (SSL)

Pre-training with Self-supervised Learning (SSL)

- LLM is pre-trained through **Self-Supervised Learning** (SSL)
- SSL is a machine learning paradigm where models learn from **unlabeled** data by predicting parts of the input from other parts. Examples:
 - Predicting missing parts of text or images
 - Predicting next word in language model
- Why SSL?
 - Massive amounts of text data available online but they are **unlabeled**
 - Manual labelling is impractical at web scale
 - There are a lot of knowledge embedded within the data itself

The Workflow of SSL

- Uses a **pretext task** to train a pre-trained model
- Pretext task must demand the model to learn **generic high-level** features
- Training set for the pretext task are **not manually labelled**
- Labels are extracted **from the data itself** based on the pretext task
- Aims to train a pre-trained model with high **generalization** capability



Self-Supervised Learning vs Other Learning Paradigms

	Unsupervised	Self-Supervised	Supervised
Goal	Discover underlying structure / pattern	Learn generic representations	Predict targeted variables
Tasks	Clustering, dimensionality reduction, anomaly detection	Masked Language Modeling (MLM), Causal Language Modeling (CLM), etc.	Classification, regression, object detection, etc.
Input Data	Unlabelled data	Unlabelled data	Labelled data
Required Labels	No labels required.	Labels are generated from data. No external labels.	Labels are manually annotated. Expensive to obtain, time-consuming

SSL Method 1: Masked Language Modeling (MLM)

- Randomly **mask** tokens in input text. Train the language model to predict masked tokens.

Randomly mask
10% of data

A quick [mask] fox jumps over the [mask] dog

Predict

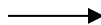
brown lazy

- Model sees **full context** (past + future) – captures **richer** context.
- Model is **bidirectional** (better for **understanding**)
- Better for tasks needing **full-context understanding**, e.g., Named-Entity Recognition (NER) or Question Answering (QA)
- Used in BERT-style encoder, e.g., **BERT**, **RoBERTa** and **ALBERT** papers

SSL Method 2: Causal Language Modeling (CLM)

- Predict **next token** given previous tokens (autoregressive).

*Nothing is impossible. Even the
word impossible says I'm possible.*



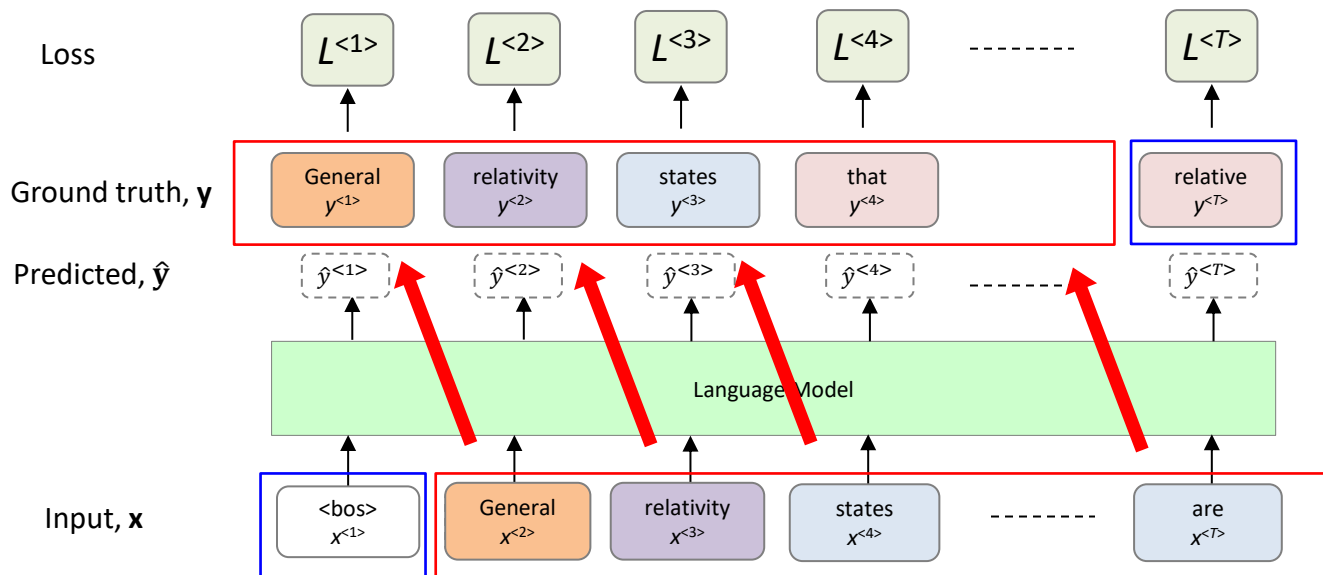
t = 1: **Nothing**
t = 2: Nothing **is**
t = 3: Nothing is **impossible**
t = 4: Nothing is impossible.
t = 5: Nothing is impossible. **Even**
...

- Processes text **sequentially**.
- Only **past** tokens are visible (no future context)
- Ideal for **generative** tasks
- **Less context-aware** than MLM
- Used in **decoder-only** model, e.g., **GPT** or **LLaMA**.

Pre-training: Causal Language Modeling (CLM)

Training in Causal Language Modeling

- For training, use **teacher forcing** where we right-shift the **ground truth** $x^{<t>} = w^{<t-1>}$ as the input to the network.



- LM predicts the **whole** output sequence $\hat{y}^{<1>} \dots \hat{y}^{<T>}$ given the **whole** input sequence $x^{<1>} \dots x^{<T>}$ $\rightarrow$ forward function can be **parallelized**!

Generating the Training Set

- The training samples are generated automatically from the raw text data
- For each sample:
 - x : randomly sample a block of text with length T

```
start_positions = sample B positions from (0, len(data) - block_size - 1]
x               = for each position, get the block data (start:start+block_size)
```

- y : the block of text at position x shifted by 1

```
y = for each position, get the block data (start+1:start+block_size+1)
```

BERT, which stands for Bidirectional Encoder Representations from Transformers, is based on Transformers, $x^{(1)}$ a deep learning model in which every output element is connected to every input element $y^{(1)}$ and the weightings between them are dynamically calculated based upon their connection. $x^{(2)}$ $y^{(2)}$

X_batch	Y_batch
a deep learning model	deep learning model in
to every input element	every input element and

Training a SOTA Model

- Training a SOTA model is costly.
- Example: LLaMA3 405B
 - Data: 15.6 trillion tokens
 - Number of parameters = 405 billion
 - FLOPs = $6 * \text{training_size} * \text{num_params} = 6 * 15.6^{12} * 405^9 = \sim 3.8^{25}$
 - Number of compute: 16,000 H100
 - Time = $3.8^{25} / (400^{12} * 3600) = 26\text{M GPU hour} / (16e^3 * 24) = 70\text{ days}$
(from paper: $\sim 30\text{M}$)
 - Estimated cost = rented computer + salary
 - = $\sim \$2/\text{h} * 26\text{Mh} + 500\text{k}/\text{y} * 50\text{ employees (Mh: million hours)}$
 - = $\$52\text{M} + \25M
 - = $\$75\text{M}$

Pre-training: Evaluation

Evaluation of LLMs

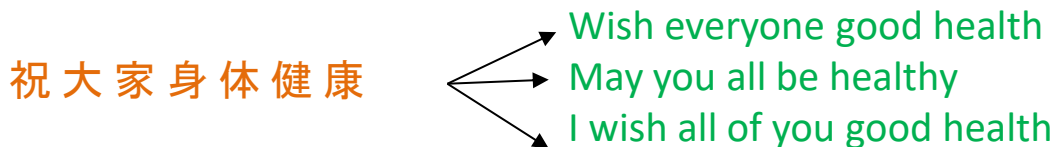
Evaluation	Description	Examples	Strength	Weaknesses
Intrinsic Evaluation	Evaluate the model's internal (linguistic) abilities on core language tasks (e.g., predicting the next token).	BLEU, ROUGE, perplexity, etc.	Automated and quick.	May not reflect real-world usefulness.
Extrinsic Evaluation	Task-based evaluation. Evaluate the model on specific downstream task.	Sentiment classification, Text summarization, Question answering	Directly tied to practical use cases.	Requires diverse benchmarks for generalizability.
Human Evaluation	Subjective quality of the model output by human evaluators	Fluency, Helpfulness, Clarity	Captures nuanced aspects of quality.	Expensive, time-consuming and subjective.
Ethical and Safety Evaluation	Assessing potential risks and biases	Bias & Fairness, Harmful Content, Truthfulness & misinformation, Privacy & security Alignment & Safety	Ensures the model follows ethical guidelines.	Complex to quantify, hard to defined the standards

Intrinsic Evaluation

- Measures how well a language model performs on **core language tasks** using **automated metrics**.
- Does not require downstream application or human judgement
- Example:
 - **Perplexity**: how well the model predicts the next word
 - **BLEU / ROUGE / METEOR**: measures the overlap with reference outputs. Used in tasks like machine learning translation or summarization
 - **Accuracy / F1 Score**: For classification or token-level tasks
- Strength: **Fast, cheap and reproducible**
- Limitations: **Does not reflect real-world usefulness or human preference**

Intrinsic Evaluation: The BLEU Score

- There could exist multiple right answers (references) given an input:



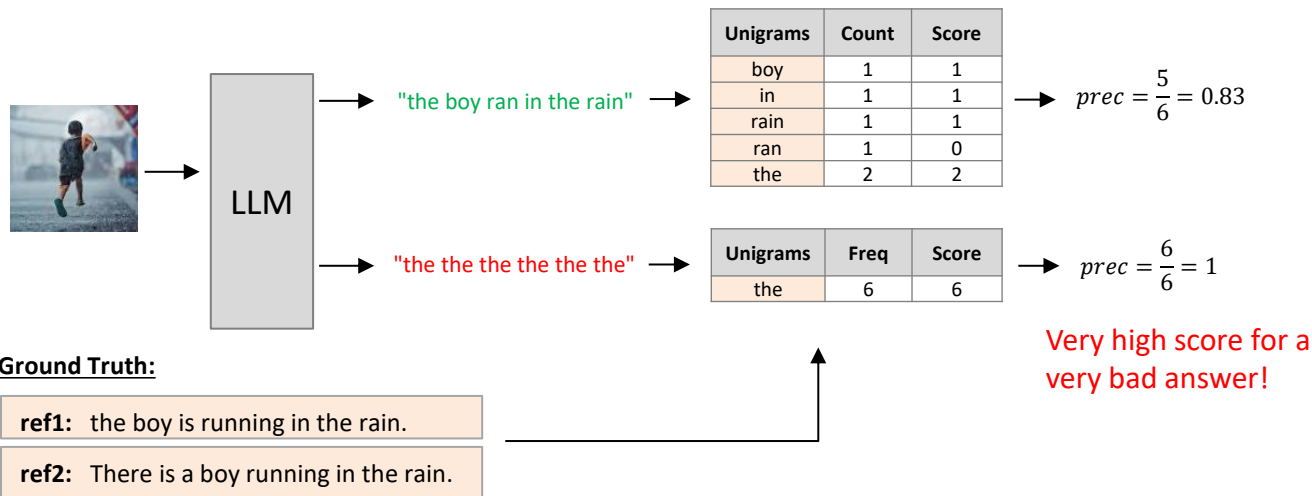
- Issues with **standard precision**:
 - Unable to **differentiate** good vs bad answers
 - Unable to evaluate tasks with **multiple** solutions.
- To solve the problem, use **BLEU**:
 - Modifies the standard precision to **limit** the contribution to the **maximum frequency** of **n-grams** in any reference

Intrinsic Evaluation: Issues with Standard Precision

- **Standard precision** computes the ratio of tokens in the generated sentence that are also found in any references (ground truth answers).

$$\text{prec}(\hat{y}) = \frac{\sum_i \text{exist_in_any_ref}(\hat{y}_i)}{|\hat{y}|}$$

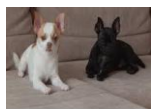
- **Issue:** High score for incorrect answer with many repetitive ground truth tokens.



Intrinsic Evaluation: Modified Precision for Unigrams

- To prevent over-counting, cap the count of each **unigram** in the generated sentence at the maximum count of any reference sentence.

$$p_1 = \frac{\sum_{w \in \text{unigram}} \min(\text{count}_{\text{LLM}}(w), \max(\text{count}_{\text{gt}}(w)))}{\sum_{w \in \text{unigram}} \text{count}_{\text{LLM}}(w)}$$



LLM

"the dog dog dog the
dog the"

Unigrams	Count
dog	4
the	3

Ground Truth:

ref1: The two dogs are sitting on the sofa.

ref2: There are one white dog and one black dog.

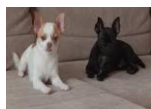
Unigrams	Count		
	ref1	ref2	Count (max)
and	-	1	1
are	1	1	1
black	-	1	1
dog(s)	1	2	2
on	1	-	1
one	-	2	2
sitting	1	-	1
sofa	1	-	1
the	2	-	2
two	1	-	1
there	-	1	1
white	-	1	1

$$p_1 = \frac{\min(4, 2) + \min(3, 2)}{4 + 3} = \frac{4}{7} = 0.5714$$

Intrinsic Evaluation: Modified Precision for Bigrams

- To prevent over-counting, cap the count of each **bigram** in the generated sentence at the maximum count of any reference sentence.

$$p_1 = \frac{\sum_{w \in \text{bigram}} \min(\text{count}_{\text{LLM}}(w), \max(\text{count}_{\text{gt}}(w)))}{\sum_{w \in \text{bigram}} \text{count}_{\text{LLM}}(w)}$$



LLM

"the dog dog dog the
dog the"

Bigrams	Count
dog the	2
dog dog	2
the dog	2

Ground Truth:

ref1: The dogs are sitting on the sofa.

ref2: There are one white dog and one black dog.

Bigrams	Count		
	ref1	Ref2	Max
and one	-	1	1
are sitting	1	-	1
are one	-	1	1
black dog	-	1	1
dogs are	1	-	1
dog and	-	1	1
on the	1	-	1
one black	-	1	1
one white	-	1	1
sitting on	1	-	1
the sofa	1	-	1
the dog(s)	1	-	1
there are	-	1	1
white dog	-	1	1

$$p_1 = \frac{\min(2, 0) + \min(2, 0) + \min(2, 1)}{2 + 2 + 2}$$

$$= \frac{1}{6} = 0.3333$$

Intrinsic Evaluation: The BLEU Score

- **BLEU** (Bilingual Evaluation Understudy) is the exponential of the average modified precision for n -grams where $n = 1 \dots N$ (Typically $N = 4$ or 5)

$$\text{bleu score} = \beta \exp\left(\sum_{i=1}^N w_i \ln(p_i)\right)$$

where

- p_i is the modified n -gram precision
- w_i is the weights (often uniform)
- β is the **brevity penalty** (penalizes translations that are too short)

$$\beta = \begin{cases} 1 & \text{if length of } \hat{y} > \text{length of } y^* \\ \exp\left(1 - \frac{\text{length of } y^*}{\text{length of } \hat{y}}\right) & \text{otherwise} \end{cases}$$

$\hat{y}$: candidate sentence

y^* : reference sentence with closest length to $\hat{y}$

- Score range is $[0,1]$, higher is better.
 - BLEU score > 0.3 : Translation is **understandable**
 - BLEU score > 0.5 : Translation is **good** and **fluent**

Extrinsic Evaluation

- Evaluates how well a model performs on different **downstream tasks** and **datasets** (e.g., summarization, question answering)
- Focuses on **real-world** performance
- Benefits:
 - Helps in **model selection** for deployment
 - Reveals **true strengths and weakness** in specific domains (code, medical, reasoning)
- Example LeaderBoards:
 - [HuggingFace](#)
 - [LMArena](#)
 - [Lambda](#)

Extrinsic Evaluation: Open LLM Leaderboard v1

https://huggingface.co/docs/leaderboards/en/open_llm_leaderboard/archive

- An **extrinsic** evaluation benchmark released by Hugging Face on June 2023
- Evaluates on 6 different tasks.
- LLM models outperform human baseline in less than 1 year.



Benchmark Tasks:

- 1. ARC (AI2 Reasoning Challenge):** Grade-school level, [multiple-choice science questions](#). Evaluates on reasoning skills and basic scientific knowledge.
- 2. HellaSwag:** Select the [best ending](#) to a given scenario given multiple options. Evaluates on common sense reasoning and situational understanding.
- 3. MMLU (Massive Multitask Language Understanding):** Contains 57 subjects from [different fields](#) like mathematics, history, law, and medicine. Evaluates the model's breadth and depth of knowledge.
- 4. TruthfulQA:** Evaluates on the model's ability to provide [truthful answers](#) to questions designed to elicit false or misleading responses. It assesses the model's resistance to generating misinformation.
- 5. Winogrande** evaluates commonsense reasoning through [pronoun resolution](#) tasks. Models must correctly identify the antecedent of a pronoun in sentences with potential ambiguity.
- 6. GSM8K (Grade School Math 8K)** tests [mathematical problem-solving skills](#) using grade school-level word problems. It assesses a model's ability to understand, reason about, and solve multi-step math problems.

Extrinsic Evaluation: Open LLM Leaderboard v2

https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard#/

- Introduced in June 2024 and includes 6 more difficult tasks.

Benchmark Tasks:

1. **MMLU-Pro:** An enhanced version of the MMLU dataset covering [multiple fields](#). Each choices have ten choices instead of four.
2. **GPQA (Google-Proof Q&A Benchmark):** A highly challenging [knowledge dataset](#) designed by domain experts to ensure difficulty and factuality.
3. **MuSR (Multistep Soft Reasoning):** A dataset of [algorithmically generated complex problems](#), including murder mysteries and team allocation optimizations, to test reasoning and long-range context parsing.
4. **MATH (Mathematics Aptitude Test of Heuristics, Level 5 subset):** High-school level [math competition](#) problems formatted for rigorous evaluation, focusing on the hardest questions.
5. **IFEval (Instruction Following Evaluation):** Tests models' ability to follow [explicit instructions](#), using rigorous metrics for evaluation.
6. **BBH (Big Bench Hard):** A subset of 23 challenging tasks from the [BigBench dataset](#) covering [multistep arithmetic, algorithmic reasoning, and language understanding](#).

- Broader range of metrics (e.g., instruction-following capabilities, incorporates qualitative assessments and user feedbacks.)
- Wider, more diverse of datasets that reflect real-world applications

Extrinsic Evaluation: Open LLM Leaderboard v2

https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard#/

■ Leaderboard result (Updated on 20/4)

	Rank	Type	Model	Average	IFEval	BBH	MATH	GPQA	MUSR	MMLU-PRO	CO ₂ Cost
🏆	1	◆	MaziyarPanahi/calme-3.2-instruct-78b	● 52.08 %	80.63 %	62.61 %	40.33 %	20.36 %	38.53 %	70.03 %	66.01 kg
🏆	2	○	MaziyarPanahi/calme-3.1-instruct-78b	● 51.29 %	81.36 %	62.41 %	39.27 %	19.46 %	36.50 %	68.72 %	64.44 kg
🏆	3	○	dfurman/CalmeRys-78B-Orpo-v0.1	● 51.23 %	81.63 %	61.92 %	40.63 %	20.02 %	36.37 %	66.80 %	25.99 kg
🏆	4	○	MaziyarPanahi/calme-2.4-rys-78b	● 50.77 %	80.11 %	62.16 %	40.71 %	20.36 %	34.57 %	66.69 %	25.95 kg
🏆	5	◆	huihui-ai/Qwen2.5-72B-Instruct-abliterated	● 48.11 %	85.93 %	60.49 %	60.12 %	19.35 %	12.34 %	50.41 %	76.77 kg
🏆	6	○	Qwen/Qwen2.5-72B-Instruct	● 47.98 %	86.38 %	61.87 %	59.82 %	16.67 %	11.74 %	51.40 %	47.65 kg
🏆	7	○	MaziyarPanahi/calme-2.1-qwen2.5-72b	● 47.86 %	86.62 %	61.66 %	59.14 %	15.10 %	13.30 %	51.32 %	29.50 kg
🏆	8	◆	newsbang/Homer-v1.0-Qwen2.5-72B	● 47.46 %	76.28 %	62.27 %	49.02 %	22.15 %	17.90 %	57.17 %	29.55 kg
🏆	9	○	ehristoforu/qwen2.5-test-32b-it	● 47.37 %	78.89 %	58.28 %	59.74 %	15.21 %	19.13 %	52.95 %	29.54 kg
🏆	10	◆	Saxo/Linkbricks-Horizon-AI-Avengers-V1-32B	● 47.34 %	79.72 %	57.63 %	60.27 %	14.99 %	18.16 %	53.25 %	7.95 kg

Human Evaluation

- **Human** judges assess model outputs based on quality, relevance, coherence, etc.
- **Benefit:**
 - Capture subtleties that automatic metric misses
 - Useful for open-ended tasks (e.g., summarization, dialogue, reasoning)
 - Provides a realistic measure of user-perceived performance
- **Weakness:**
 - **Subjective:** human judgments can vary across annotators
 - **Inconsistent:** lack of domain knowledge, ratings may drift over time sometimes due to fatigue
- **Platforms:**
 - [LMArena](#) (previously chat arena)
 - [LiveBench](#) (daily human comparisons)

Human Evaluation: LM Arena

<https://lmarena.ai/>

- A human asks any question to two anonymous AI chatbots and vote for best model.
- Ranks models based on the **Elo rating system**
- Example: **LM Arena** by UC Berkeley & LM Arena


Choose two models to compare

chatgpt-4o-latest-20241120

Model A

yl-lightning

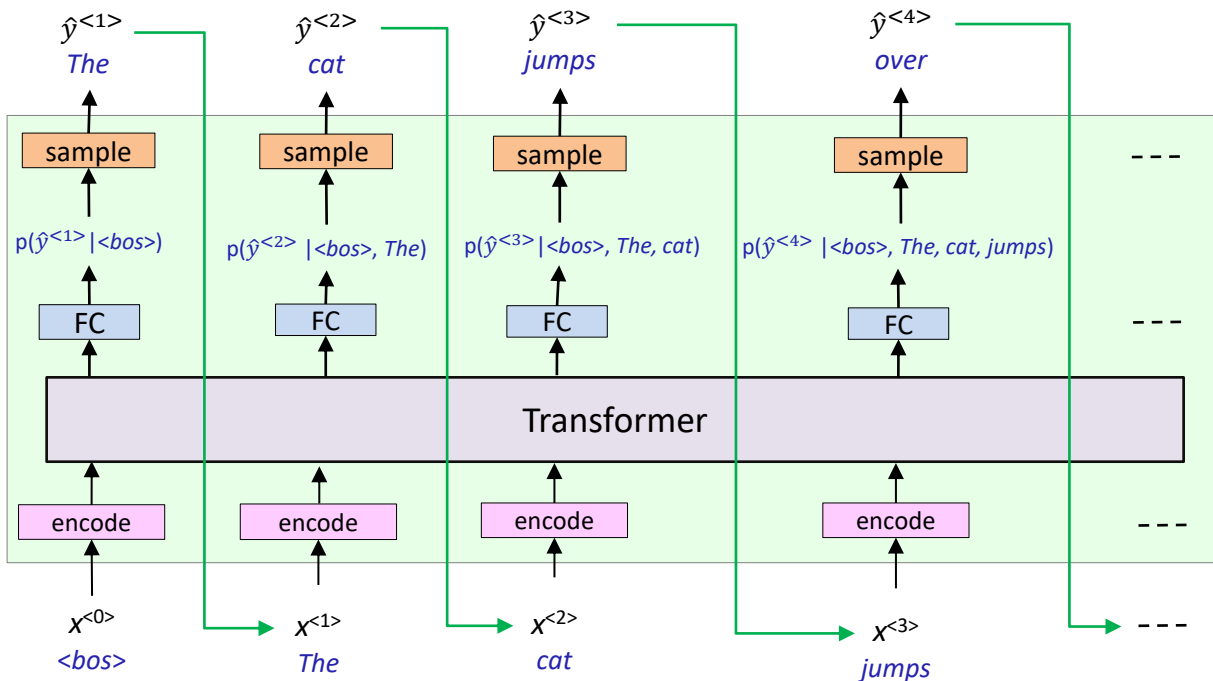
Model B

Rank* (UB)	Rank (StyleCtrl)	Model	Arena Score	95% CI	Votes	Organization	License
1	1	Gemini-2.5-Pro-Exp-03-25	1439	+7/-5	9013	Google	Proprietary
2	2	ChatGPT-4o-latest-(2025-03-26)	1407	+6/-6	8261	OpenAI	Proprietary
2	4	Grok-3-Preview-02-24	1402	+5/-3	14849	xAI	Proprietary
2	2	GPT-4.5-Preview	1398	+5/-6	14520	OpenAI	Proprietary
2	4	Gemini-2.5-Flash-Preview-04-17	1392	+10/-13	3325	Google	Proprietary
5	9	Gemini-2.0-Flash-Thinking-Exp-01-21	1380	+4/-3	26309	Google	Proprietary
5	5	Gemini-2.0-Pro-Exp-02-05	1380	+5/-4	20127	Google	Proprietary
5	4	DeepSeek-V3-0324	1372	+9/-6	5888	DeepSeek	MIT
9	6	DeepSeek-R1	1358	+4/-5	16077	DeepSeek	MIT
9	14	Gemini-2.0-Flash-001	1354	+3/-4	22201	Google	Proprietary
10	6	o1-2024-12-17	1350	+3/-3	28990	OpenAI	Proprietary
11	14	Gemma-3-27B-it	1342	+5/-6	9976	Google	Gemma

Inference Mode

Inference Mode

- During inference mode, text is generated **recursively** (no parallelization). At time step t , the model generates the **next** token $\hat{y}^{<t>}$ given the **previous** token sequence $\hat{y}^{<1>} \dots \hat{y}^{<t-1>}$.



The Inference Algorithm

Inference Algorithm for Auto-regressive Language Model

Input:

- prompt: initial token sequence $\mathbf{x} = (x^{<1>}, \dots, x^{<T_x>})$
- context_length: max tokens model can attend to
- max_length: maximum generation length

Output :

- Generated sequence: $\hat{\mathbf{y}} = (\hat{y}^{<1>}, \dots, \hat{y}^{<T_y>})$

1. Set $t = 1$
2. Set **start** index: $\text{start} = \max(1, T_x + t - \text{context_length} + 1)$
3. Perform **inference** on LM to get

$$p(\hat{y}^{<t>} | x^{<\text{start}>}, \dots, x^{<T_x+t>})$$

4. **Sample** the token (use `torch.multinomial`)

$$\hat{y}^{<t>} \sim p(\hat{y}^{<t>} | x^{<\text{start}>}, \dots, x^{<T_x+t>})$$

5. **Append** predicted token $\hat{y}^{<t>}$ to input sequence such that:

$$\mathbf{x} = (x^{<1>}, \dots, x^{<T_x+t>}) = (x^{<1>}, \dots, x^{<T_x>}, y^{<1>}, \dots, y^{<t>})$$

6. Set $t = t + 1$
7. **Repeat** Step 2 to 6 until $t \geq \text{max_length}$ or $\hat{y}^{<t>} = \text{<EOS>}$

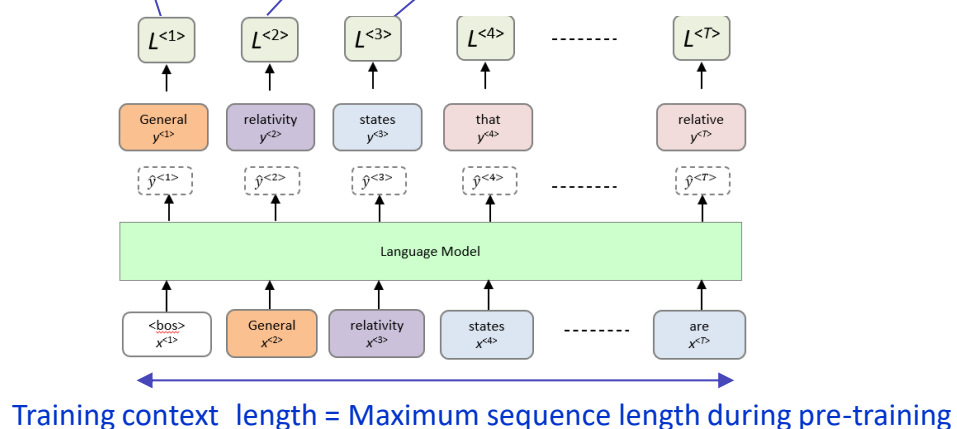
Training Context Length

- **Training Context Length** = maximum input sequence length used during initial, massive pre-training phase
- Inference with input sequence longer than training context length → **degraded** performance
- Generalizes well to shorter sequences.

Learning to predict on sequence with $T = 1$

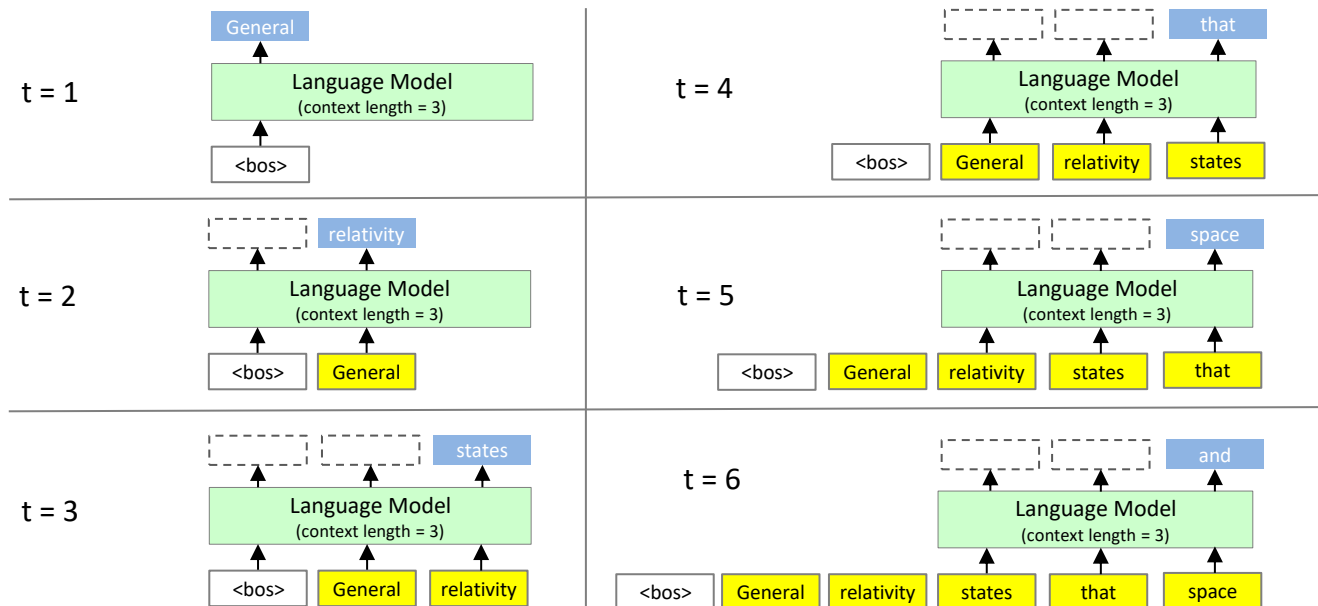
Learning to predict on sequence with $T = 2$

Learning to predict on sequence with $T = 3$



Trimming Long Sequences to Context Length

- Model cannot "remember" information outside the window — if input exceeds the training context length, older tokens are **truncated**.
- Example (context length = 3):



Effective Context Length

- Model often struggle to use their full training length in practice.
- **Context Length:** maximum number of tokens that a model can process at once at **inference** time.
 - Includes the **user prompts**, **context data** from RAG, **conversation history**, **system instructions** and appended **model outputs**.
 - Architectural + deployment constraints — defines the operational limit of the model.
 - Measured in **tokens** (1 token $\approx$ 0.75 words)
- Longer context length allows **long conversations** without forgetting earlier details and processing of long documents.

Factors affecting Context length

▪ **Model architecture:**

- Limited GPU / TPU memory or KV cache size — core limitation to context length
- Memory and compute cost grows quadratically with number of tokens n : $O(n^2)$.

▪ **Positional Encoding:**

- Absolute position embeddings → fixed maximum length
- **Rotary / relative position embedding (RoPE)** → flexible, can extrapolate to longer contexts but poor extrapolation → degraded performance.

▪ **Training Context Window:**

- Model is trained with a maximum sequence length → the model is only trained to reason within that window
- Longer input at inference leads to degraded performance

▪ **Attention Optimization:**

- Mechanism to extend attention, e.g., **sliding window** attention, **sparse / block** attention, **memory-augmented** attention.

Inference: Decoding Strategies

Decoding Strategies

- Decoding strategies are methods used to generate text from a language model's output probabilities.
- Common decoding strategies:
 - **Greedy decoding:** Picks the most probable token at each step (repetitive and dull)
 - **Top- k sampling:** Samples from the top k most likely tokens
 - **Top- p sampling:** Samples from the smallest possible set of tokens whose cumulative probability is $\geq p$.
 - **Temperature:** Controls randomness. Lower = more deterministic. Higher = more diverse.
 - **Beam Search:** Explores multiple paths and picks the most full sequence. Mainly used in machine translation.

Greedy Sampling

- A **deterministic** decoding strategy that selects the **highest probability** token at each time step. At time step t , choose:

$$y_t = \arg \max_y P(\hat{y} | \hat{y}_1, \hat{y}_2, \dots \hat{y}_{t-1}, x_t)$$

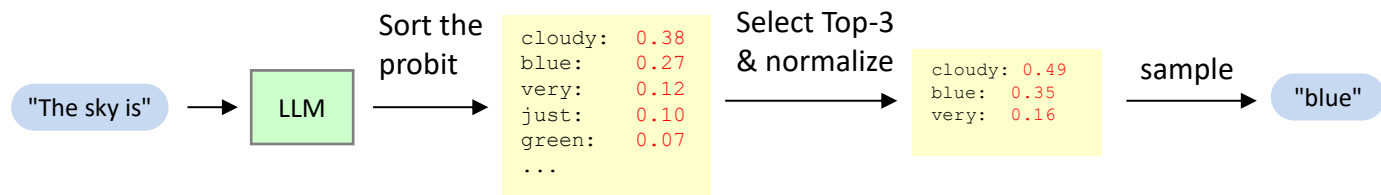
- **Simple and Fast:** Only one token chosen per step, **fast** and **lightweight**, minimal memory and compute requirement
- **Effective** for **short, high-confidence sequence** where the model's probit is sharply peaked (one token is clearly the choice).
- **Deterministic:** Always delivers the same output, **lacks diversity**, may generate **repetitive** or **generic** text (e.g., "I don't know" loops)
- **Myopic:** no lookahead; may generate **sub-optimal sequences** since local optimal choice at each step may not lead to globally best sequence

Balancing Creativity and Coherence

- Greedy decoding → deterministic but repetitive
- Random sampling → diverse but incoherent
- **Solution:**
Balance creativity and coherence with **Top- k** and Top- n (**nucleus**) sampling

Top-k Sampling

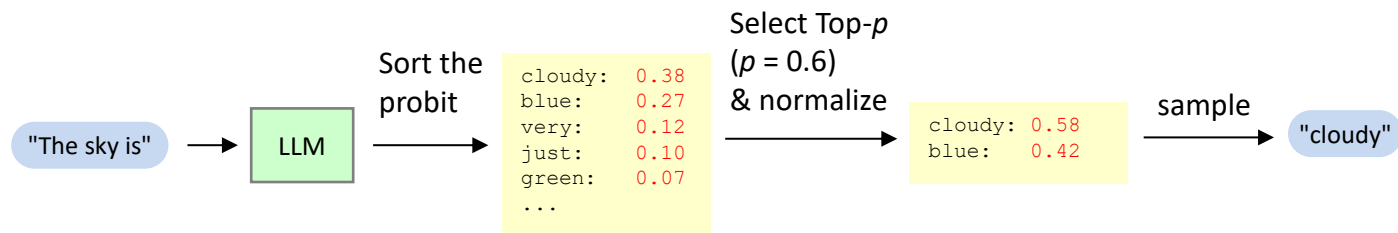
- Top-k Sampling:
 - Sort tokens by probability
 - Select the top k most likely tokens
 - Re-normalize probabilities → sample from this subset



- More **diverse** than greedy decoding
- Tunable** with k
- Sensitive to k** —output may be destroyed if top k are all bad

Top- p (Nucleus) Sampling

- Top- p Sampling:
 - Sort tokens by probability
 - Select smallest set where their cumulative probability $\geq p$
 - Re-normalize probabilities $\rightarrow$ sample from this subset

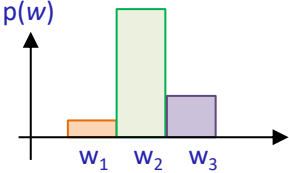
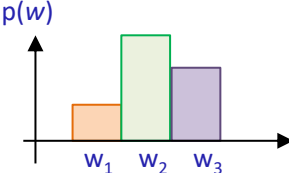
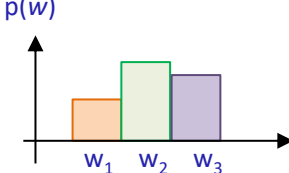


- Adapts** to uncertainty. If distribution is sharp, select fewer tokens. If flat, select more tokens
- Better than Top- k for **long-form** creativity
- Requires tuning p (typically 0.7 – 0.95)

Temperature Decoding

- The **Temperature** (T) hyperparameter **sharpens** or **flattens** the probability distribution before sampling.
- Applied to **logits** before applying softmax

$$p'(w_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

$T < 1$	$T = 1$	$T > 1$
 Distribution becomes sharper (more deterministic)	 No changes	 Distribution becomes flatter (more random)
Favours high-probability tokens	No changes	Increases diversity

- Can be used alongside other decoding strategies such as top- k or top- p sampling, repetition penalty, and beam search.

Repeat Penalty

- The **repeat penalty** discourages **repetitive** tokens by **reducing** the probabilities of tokens generated within a certain **context window**.
- Applied to **logits** before softmax or sampling

$$z'_i = \frac{z_i}{\text{repeat_count} * \text{penalty_factor}}$$

Parameter	Typical Value	Effect
Penalty factor	1.1 – 1.5	Higher = more aggressive suppression
Context Window	10 – 100 tokens	How far back to check for repeats

- Too high penalty may cause **nonsensical** outputs
- May be combined with temperature sampling and Top- k or Top- p sampling

Issues with single-step decoding methods

- **Myopic Decisions**

- Only consider current step's probability
- Ignores long-term sequence quality

- **Suboptimal Sequences**

- Greedy method may lead to unnatural or incomplete sentences
- Sampling can produce incoherent outputs

- **Lack of diversity**

- Greedy method produces the same output each time
- Poor exploration of alternative plausible sequences

Beam Search

- **Beam Search** keeps the **top k** most probable partial sequences (**beams**) at each time step instead of just one.
 - k is the **beam width**: controls the trade-off between quality and computational cost
 - At each step, all possible next tokens are considered for each beam, and the top k sequences are kept
- Advantages:
 - **Less myopic** –looks ahead multiple steps before deciding
 - **More coherent output** – keeps better sequences even if initial token is not the top choice
 - **Controllable diversity** – Higher beam width leads to higher diversity
- Mainly used in tasks that generates the **full sentence** in a single inference, e.g., machine translation.

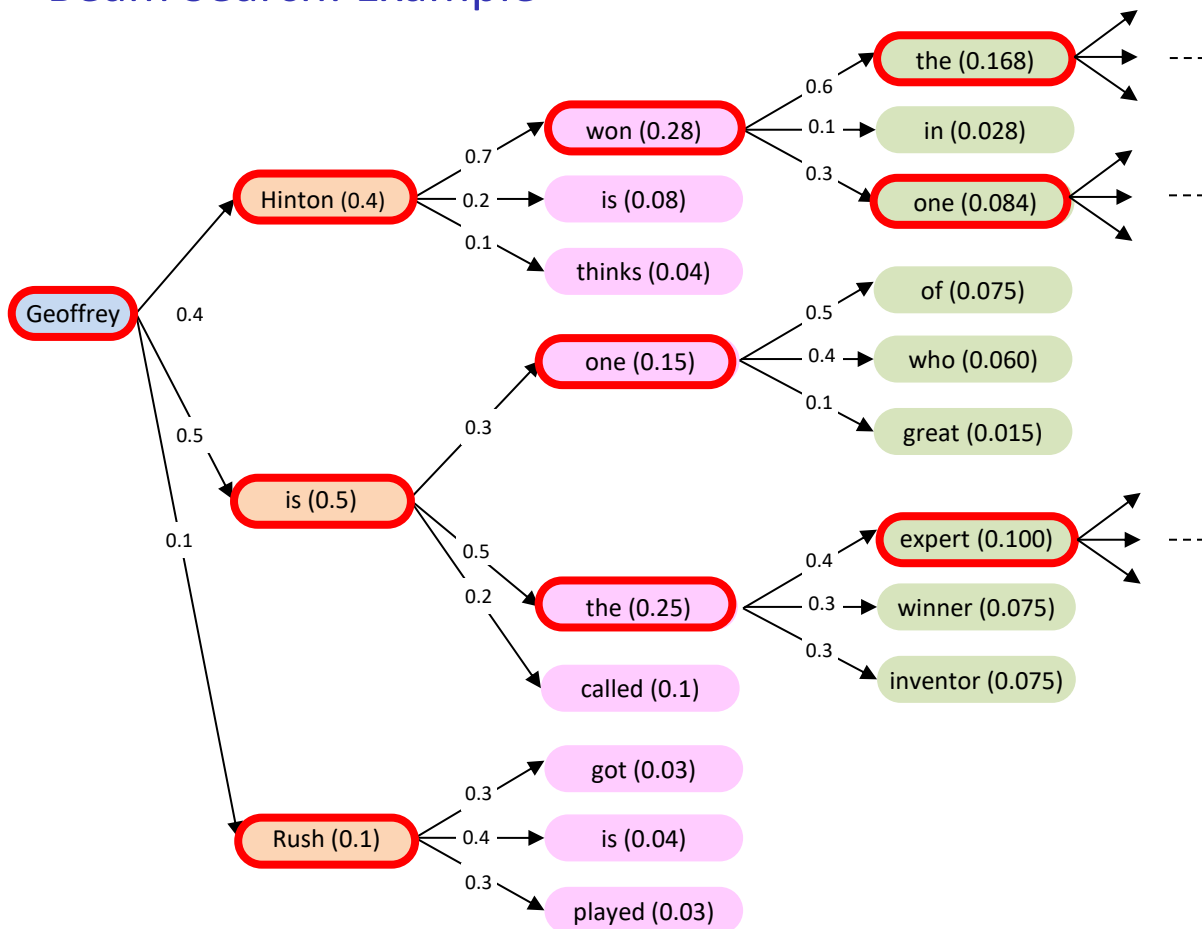
The Beam Search Algorithm

Input : start_token, end_token, max_length, beam_size

Output : sequence with the highest beam score

1. **Initialization:** Start with a single beam containing only the *start_token* and an initial score of 0 (log-probability of 1.0).
2. **Beam Expansion:** *Expand* all beams by using the model to predict the probabilities of the next token. Each token is appended to the corresponding input beam. new score = current beam's score + log probability of the new token.
3. **Pruning:** Keep only the top *beam_size* beams with the highest scores. This avoids exponential growth in the number of beams.
4. **Repeat:** Repeat step 2 and 3 until the algorithm reaches the *end_token* for a beam or hits the *max_length* limit.
5. **Selection of Best Sequence:** The sequence with the *highest score* among the beams is returned as the final output.

Beam Search: Example



*Thank you
for your attention*